

Optimal Projectile Interception: A Resource Allocation Problem

Thiago da Cunha Vasco

August 13, 2025

Abstract

This report details a side project I briefly worked on near the end of my masters degree. During the time of writing, discussions regarding anti-missile defense systems were circulating around the news following Finland's entrance into NATO and President Trump's proposal for a "golden dome" which would protect the American mainland from incoming missile attacks. This prompted me to ponder how one could construct a decision process for deciding which incoming projectiles one should intercept. The result is a resource allocation problem which can be solved using either stochastic dynamic programming or value iteration. Needless to say, the problem posed in this report is a massive simplification of the true interception problem one would have to solve when developing a real-world anti-missile system. Nonetheless, I still find my results interesting and worth sharing.

1 The Single-Period Problem

We begin our discussion by defining variables and constructing a model which solely concerns a single instant in time. Suppose our sensors detect n projectiles in the air, where each projectile is projected to land in one of l zones. Let c^j be the cost we incur if any projectile $i \in [n]$ lands in zone $j \in [l]$. Furthermore, suppose we have m interceptors which can be launched to shoot down projectiles. The cost associated with launching an interceptor is c_{DPI} . When an interceptor is launched to intercept a projectile, it is not guaranteed to successfully hit its target; we define p to be the probability of a successful hit. If m_i interceptors targeting projectile i are launched, the probability that all of them miss (and the projectile survives) is $(1 - p)^{m_i}$. Assuming no interception, the cost imposed by projectile i is given by the random variable C_i , where

$$C_i = \begin{cases} c^1 & \text{w.p. } p_i^1 \\ c^2 & \text{w.p. } p_i^2 \\ \vdots & \\ c^l & \text{w.p. } p_i^l, \end{cases} \quad (1)$$

and p_i^j is the probability that projectile i lands in zone j . The expected cost imposed by projectile i is then simply $\bar{c}_i \triangleq \sum_{j=1}^l p_i^j c^j$. If we take into consideration not only where projectile i is likely to land, but also how likely it is to survive m_i interceptors, we would say that the *effective cost* imposed by projectile i is

$$C_i^{\text{eff}} = \begin{cases} 0 & \text{w.p. } p_i^{\text{hit}} \\ c^1 & \text{w.p. } p_i^{\text{miss}} p_i^1 \\ \vdots & \\ c^l & \text{w.p. } p_i^{\text{miss}} p_i^l, \end{cases} \quad (2)$$

where $p_i^{\text{hit}} = 1 - (1 - p)^{m_i}$ and $p_i^{\text{miss}} = (1 - p)^{m_i}$. Our goal then is to choose how many interceptors to allocate to each projectile such that the expected cost is minimized, and subject to constraints. That is, we have a resource allocation problem of the form

$$\begin{aligned} \min_{m_1, \dots, m_n} \quad & \mathbb{E}\left[\sum_{i=1}^n (C_i^{\text{eff}} + c_{DPI} \cdot m_i)\right] \\ \text{s.t.} \quad & \sum_{i=1}^n m_i \leq m, \quad m_i \in \mathbb{N} \cup \{0\}. \end{aligned} \tag{3}$$

It is easy to show that $\mathbb{E}[\sum_{i=1}^n (C_i^{\text{eff}} + c_{DPI} \cdot m_i)] = \sum_{i=1}^n (\bar{c}_i(1 - p)^{m_i} + c_{DPI} \cdot m_i)$, and therefore the problem becomes

$$\begin{aligned} \min_{m_1, \dots, m_n} \quad & \sum_{i=1}^n (\bar{c}_i(1 - p)^{m_i} + c_{DPI} \cdot m_i) \\ \text{s.t.} \quad & \sum_{i=1}^n m_i \leq m, \quad m_i \in \mathbb{N} \cup \{0\}. \end{aligned} \tag{SP}$$

The integer program **SP** is the single-period version of the optimal intercept problem. In the following section we will discover a simple algorithm for solving it. As we will soon see, a key ingredient for solving **SP** to optimality is the fact that $g_i(x) \triangleq \bar{c}_i(1 - p)^x + c_{DPI} \cdot x$ is convex in x .

1.1 A Solution Method for the Single-Period Problem

Suppose we have convex functions $f_i : \mathbb{R} \rightarrow \mathbb{R}$ for $i = 1, \dots, n$. Our goal is to solve

$$\begin{aligned} \min_{m_1, \dots, m_n} \quad & \sum_i f_i(m_i) \\ \text{s.t.} \quad & \sum_{i=1}^n m_i \leq m, \quad m_i \in \mathbb{N} \cup \{0\}. \end{aligned} \tag{SICIF}$$

We will show that **SICIF** is solved to optimality by Algorithm 1. The basic idea behind the algorithm is to increment m_i by one if the difference $f_i(m_i + 1) - f_i(m_i)$ is negative and smaller than $f_{i'}(m_{i'} + 1) - f_{i'}(m_{i'})$ for all other $i' \neq i$. The algorithm terminates either when all of the differences are positive, or when $\sum_i m_i = m$. Let $f^*(\mu)$ be the optimal objective function value for **SICIF** when $m = \mu$. Algorithm 1 returns the array **m_list** which contains the optimal integer solution $m_1, \dots, m_n \in \mathbb{N} \cup \{0\}$, and also the array **obj_values** which contains optimal objective values $f^*(0), f^*(1), \dots, f^*(m)$ for when $\mu = 0, 1, \dots, m$. The latter is computed with little added expense, and will come in handy when using dynamic programming to solve the multi-period problem.

Algorithm 1: Sum of Independent Convex Integer Functions (SICIF)

Data: $m \geq 0$, $n > 0$, convex functions $f_i(\cdot)$ for $i \in [n]$

```
1 m_list  $\leftarrow$  zeros( $n$ );
2 curr_m  $\leftarrow$  0;
3 obj_values  $\leftarrow$  zeros( $m + 1$ );
4 obj_values[0]  $\leftarrow$   $\sum_i f_i(0)$ ;
5 deltas  $\leftarrow$  an  $n$ -long array where index  $i$  is given by  $f_i(1) - f_i(0)$ ;
6 sorted_idx  $\leftarrow$  argsort(deltas);
7 if  $m = 0$  then
8   return m_list, obj_values;
9 end
10 while deltas[0] < 0 do
11   i_min  $\leftarrow$  sorted_idx[0];
12   m_list[i_min]  $\leftarrow$  m_list[i_min] + 1;
13   curr_m  $\leftarrow$  curr_m + 1;
14   obj_values[curr_m]  $\leftarrow$  obj_values[curr_m - 1] + deltas[0];
15   if curr_m =  $m$  then
16     break;
17   end
18   deltas[0]  $\leftarrow$   $f_{i\_min}(m\_list[i\_min] + 1) - f_{i\_min}(m\_list[i\_min])$ ;
19   swap_idx  $\leftarrow$  argsort(deltas);
20   deltas  $\leftarrow$  deltas[swap_idx];
21   sorted_idx  $\leftarrow$  sorted_idx[swap_idx];
22 end
23 obj_values[curr_m+1:end]  $\leftarrow$  obj_values[curr_m];
24 return m_list, obj_values;
```

We will now go over the runtime of the algorithm and the proof of optimality.

Proposition 1. *Algorithm 1 terminates in finite time.*

Proof. Note how at each iteration of the while loop (line 10 to line 21), we have that $\sum_i m_i$ is incremented by 1. If at any iteration we have that $\mathbf{deltas}[i] \geq 0$ for all $i \in [n]$, the algorithm terminates. Otherwise, the algorithm will terminate once $\sum_i m_i = m$. $\square$

Now that we are certain the algorithm terminates in finite time, we can now study its runtime.

Proposition 2. *Algorithm 1 terminates in $O((n + m) \log n)$ time.*

Proof. At line 6 the array $\mathbf{deltas}$ is sorted which takes $O(n \log n)$ time. Every procedure from line 11 to line 18 is $O(1)$. On line 19 we once again sort $\mathbf{deltas}$, but because the array at this point is already sorted save the first entry, this amounts to binary search which is $\log n$. The while-loop will iterate for at most m iterations, which means that line 10 to line 22 will terminate in $O(m \log n)$. Combining the runtime of line 6 with the runtime of the while-loop gives us $O((n + m) \log n)$. $\square$

Before we tackle the proof of optimality, we require two simple preliminary results.

Proposition 3. *Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function. Then we have that for all $k \in \mathbb{Z}$,*

$$f(k) - f(k - 1) \leq f(k + 1) - f(k). \quad (4)$$

Proof. Because f is convex, we have that

$$f(\lambda(k-1) + (1-\lambda)(k+1)) \leq \lambda f(k-1) + (1-\lambda)f(k+1) \quad (5)$$

for all $\lambda \in [0, 1]$. Choose $\lambda = \frac{1}{2}$. Then we have

$$f(k) = f\left(\frac{1}{2}(k-1) + \frac{1}{2}(k+1)\right) \leq \frac{1}{2}f(k-1) + \frac{1}{2}f(k+1). \quad (6)$$

If we multiply both sides by 2 and rearrange we get $f(k) - f(k-1) \leq f(k+1) - f(k)$. $\square$

Proposition 3 shows the connection between the convexity of f_i , and the diminishing returns of continually incrementing m_i by one. That is, we have $f_i(1) - f_i(0) \leq f_i(2) - f_i(1) \leq f_i(3) - f_i(2) \leq \dots$. So the first increment will decrease the objective more than the second increment, and the second increment will decrease the objective more than the third increment, etc.

Proposition 4. Let m_i^k denote that value of m_i at the end of iteration k of the while-loop in Algorithm 1, where $k = 1, \dots, m$. So at the end of iteration k we have $\sum_i m_i = \sum_i m_i^k = k$. Then we have that for any i such that $m_i^k \geq 1$,

$$f_i(m_i^k - p + 1) - f_i(m_i^k - p) \quad \text{for } p = 1, \dots, m_i^k \quad (7)$$

is smaller than or equal to

$$f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1) \quad \text{for } q = 1, 2, \dots \quad (8)$$

for any $i, i' \in [n]$.

Proof. If $i = i'$, then the above statement simply follows from proposition 3. So suppose $i \neq i'$. We should note that the difference $f_i(m_i^k - p + 1) - f_i(m_i^k - p)$ corresponds to an increment in m_i which occurred in a past iteration, whereas the difference $f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1)$ corresponds to an increment in $m_{i'}$ which has not yet occurred. Fix the integer p and define k' to be the iteration where m_i is incremented from $m_i^k - p$ to $m_i^k - p + 1$. Because at iteration k' , m_i was chosen to be incremented and not $m_{i'}$, by Algorithm 1 this implies that at the beginning of iteration k' ,

$$f_i(m_i^{k'-1} + 1) - f_i(m_i^{k'-1}) \leq f_{i'}(m_{i'}^{k'-1} + 1) - f_{i'}(m_{i'}^{k'-1}). \quad (9)$$

Because k' is an earlier iteration than k , we have that $m_{i'}^{k'-1} \leq m_{i'}^k$. So then

$$\begin{aligned} f_i(m_i^k - p + 1) - f_i(m_i^k - p) &= f_i(m_i^{k'-1} + 1) - f_i(m_i^{k'-1}) \\ &\leq f_{i'}(m_{i'}^{k'-1} + 1) - f_{i'}(m_{i'}^{k'-1}) \\ &\leq f_{i'}(m_{i'}^k + 1) - f_{i'}(m_{i'}^k) \\ &\leq f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1). \end{aligned} \quad (10)$$

The last two inequalities hold by proposition 3 because $f_{i'}$ is convex. $\square$

Proposition 4 essentially tells us that if Algorithm 1 increments m_i from a to $a+1$ before it increments $m_{i'}$ from b to $b+1$, the a -to- $(a+1)$ increment in m_i leads to a larger decrease in the objective function than the b -to- $(b+1)$ increment in $m_{i'}$. Finally, we show that Algorithm 1 is optimal.

Proposition 5. *Algorithm 1 returns the optimal solution for SICIF.*

Proof. Suppose we arrive at the very end of iteration k of the while-loop in Algorithm 1. We then have that $\sum_i m_i^k = k$. Now let $m'_1, \dots, m'_n$ be any other solution such that $\sum_i m'_i \leq k$.

First we will handle the case where $\sum_i m'_i < k$. The fact that Algorithm 1 has managed to find k -many increments, all of which decrease the objective function implies that the alternative solution $m'_1, \dots, m'_n$ has yet to exploit further increments that can decrease the objective while still satisfying the inequality constraint $\sum_i m'_i \leq k$. Hence, the alternative solution is sub-optimal.

Now we handle the case where $\sum_i m'_i = k$. We partition the set of indices $\{1, 2, \dots, n\}$ using three mutually exclusive sets:

$$\begin{aligned}\mathcal{I} &= \{i \in [n] : m'_i > m_i^k\} \\ \mathcal{D} &= \{i \in [n] : m'_i < m_i^k\} \\ \mathcal{R} &= \{i \in [n] : m'_i = m_i^k\}\end{aligned}\tag{11}$$

The set $\mathcal{I}$ corresponds to indices where the alternative solution has larger m_i relative to the solution given by Algorithm 1. Likewise, the set $\mathcal{D}$ corresponds to indices where the alternative solution has smaller m_i relative to the algorithm solution. Lastly, the set $\mathcal{R}$ corresponds to indices where both the alternative solution and the algorithm solution give the same value to m_i . Because $\mathcal{I} \cup \mathcal{D} \cup \mathcal{R}$ forms a partition of $[n]$, we have that

$$\sum_i m'_i = \sum_{i \in \mathcal{I}} m'_i + \sum_{i \in \mathcal{D}} m'_i + \sum_{i \in \mathcal{R}} m'_i = k,\tag{12}$$

and also that

$$\sum_i m_i^k = \sum_{i \in \mathcal{I}} m_i^k + \sum_{i \in \mathcal{D}} m_i^k + \sum_{i \in \mathcal{R}} m_i^k = k.\tag{13}$$

Because $\sum_{i \in \mathcal{R}} m'_i = \sum_{i \in \mathcal{R}} m_i^k$ we then get

$$\sum_{i \in \mathcal{I}} (m'_i - m_i^k) = \sum_{i \in \mathcal{D}} (m_i^k - m'_i) \triangleq \delta.\tag{14}$$

So the alternative solution $m'_1, \dots, m'_n$ can be reconstructed by applying δ -many increments across variables $\{m_i^k \text{ for } i \in \mathcal{I}\}$, and applying δ -many decrements across variables $\{m_i^k \text{ for } i \in \mathcal{D}\}$. Each decrement to m_i^k for any $i \in \mathcal{D}$ increases the objective function by

$$f_i(m_i^k - p) - f_i(m_i^k - p + 1) \geq 0 \text{ for } p = 1, \dots, m_i^k.\tag{15}$$

Likewise, each increment to $m_{i'}^k$ for any $i' \in \mathcal{I}$ changes the objective function by

$$f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1) \text{ for } q = 1, 2, \dots.\tag{16}$$

By proposition 4, we have that

$$f_i(m_i^k - p + 1) - f_i(m_i^k - p) \leq f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1),\tag{17}$$

and so

$$[f_i(m_i^k - p) - f_i(m_i^k - p + 1)] + [f_{i'}(m_{i'}^k + q) - f_{i'}(m_{i'}^k + q - 1)] \geq 0. \quad (18)$$

Therefore, every time we decrement one variable in $\mathcal{D}$ and increment another variable in $\mathcal{I}$, we either increase the objective or keep it the same. Therefore, $\sum_i f_i(m_i') \geq \sum_i f_i(m_i^k)$. Because the solution $m_1', \dots, m_n'$ is arbitrary, this means that the algorithm solution $m_1^k, \dots, m_n^k$ is optimal at iteration k . $\square$

2 The Multi-Period Problem

We now consider the problem where we not only have to concern ourselves about the current incoming projectiles, but now we also have to conserve interceptors for projectiles that will be launched in future time steps. Let $T \sim P(T)$ be the total number of time steps following the current instant, and let $n_t \sim P(n_t)$ be the number of new projectiles in the air at time t . Lastly, let $\Delta_{t,i} \sim P(\Delta_{t,i})$ be a vector whose entries are the probabilities of projectile $i \in [n_t]$ hitting zones $1, \dots, l$. I leave it to the reader to specify distributions $P(T)$, $P(n_t)$, and $P(\Delta_{t,i})$ to their liking. In my example code I specify $P(n_t)$ to be a Poisson distribution with rate parameter λ , whereas $P(T)$ I specify to be either a discrete uniform distribution over integers $\{a, a+1, \dots, b-1, b\}$, or a Geometric distribution with parameter ρ . Almost certainly, one would want to specify $P(\Delta_{t,i})$ to be a Dirichlet distribution with parameter $\alpha \in \mathbb{R}^l$ (or a mixture of Dirichlets to deal with multi-modal cases).

Let $t = 0$ denote the current time instant. Once again we assume that there are n incoming projectiles, each of which will impose a random cost of C_i for $i \in [n]$ as defined by [1](#). Our goal is to minimize the expected cost not only imposed by projectiles in the current timestep, but also by projectiles that are yet to come. The multi-period problem is defined as

$$\begin{aligned} \min_{m_1, \dots, m_n} & \left\{ \sum_{i=1}^n (\bar{c}_i (1-p)^{m_i} + c_{DPI} \cdot m_i) + (1 - \rho_0) V_1(m - \sum_{i=1}^n m_i) \right\} \\ \text{s.t.} & \sum_{i=1}^n m_i \leq m, \quad m_i \in \mathbb{N} \cup \{0\}, \end{aligned} \quad (\text{MP})$$

where

$$V_t(\mu) = \mathbb{E}_{n_t} \mathbb{E}_{(\Delta_{t,i})_{i \in [n_t]}} \left[\min_{(m_{t,i})_{i \in [n_t]}} \left\{ \sum_{i=1}^{n_t} (\bar{c}_{t,i} (1-p)^{m_{t,i}} + c_{DPI} \cdot m_{t,i}) + (1 - \rho_t) V_{t+1}(\mu - \sum_{i=1}^{n_t} m_{t,i}) \right\} \right] \quad (19)$$

is the value function for time step t , and $\rho_t = P(T = t | T \geq t)$ is the probability that time step t is the final time step (conditioned on the premise that t is also the current time step).

In the next section I present a simple numerical method for finding an approximate solution to **MP** which combines monte carlo sampling and dynamic programming, and which also makes use of [Algorithm 1](#).

2.1 A Solution Method for the Multi-Period Problem

Suppose we know $V_1(\cdot)$ and our goal is to solve **MP**. Although I don't formally prove this, it should be intuitive that $V_1(\cdot)$ is monotonic decreasing and convex. Therefore, we could solve **MP** using [Algorithm 1](#) we developed in [section 1.1](#). In practice however, $V_1(\cdot)$ must be estimated. Equation [19](#) tells us that in order to estimate $V_1(\cdot)$, we must first estimate $V_2(\cdot)$ and so on. The following algorithm recursively estimates $V_t(\cdot)$ for $t = T, T-1, \dots, 1$, and then finally computes an approximate optimal solution for **MP**.

For brevity, define $g_{0,i}(z) \triangleq \bar{c}_i(1-p)^z + c_{DPI} \cdot z$ for $i \in [n]$, and also define $g_{t,i}(z) \triangleq \bar{c}_{t,i}(1-p)^z + c_{DPI} \cdot z$ for $i \in [n_t]$. In the context of **MP**, $g_{0,i}(\cdot)$ is a deterministic function since $\bar{c}_i$ is already known, and $g_{t,i}(\cdot)$ is a stochastic function since $\bar{c}_{t,i}$ is determined by sampling $\Delta_{t,i}$. We take the phrase “generate $g_{t,i}(\cdot)$ for $i \in [n_t]$ ” to mean sample $\Delta_{t,i}$ and construct $g_{t,i}(\cdot)$ for $i \in [n_t]$. Lastly, in order to deal with the fact that T , n_t , and $\Delta_{t,i}$ are all stochastic, we sample R trajectories and average their value function estimates at each time step (line 14).

Algorithm 2: Dynamic Programming with Monte Carlo

Data: $m \geq 0$, $n > 0$, $R > 0$, and convex functions $g_{0,i}(\cdot)$ for $i \in [n]$

```

1 T_arr  $\leftarrow [T_1, \dots, T_R]$  where each  $T_r \sim P(T)$ ;
2 max_T  $\leftarrow \max(T\_arr)$ ;
3 v  $\leftarrow \text{zeros}(m+1)$ ;
4 for  $t = T$  to 1 do
5   cost_per_rollout  $\leftarrow \text{zeros}((m+1, R))$ ;
6   for  $r = 1$  to  $R$  do
7     if  $t \leq T\_arr[r]$  then
8       Sample  $n_t \sim P(n_t)$ ;
9       Generate  $g_{t,i}(\cdot)$  for  $i \in [n_t]$ ;
10      -, obj_values  $\leftarrow \text{SICIF}(m, n_t + 1, [(g_{t,i})_{i \in [n_t]} \text{ and } v])$ ;
11      cost_per_rollout[:, r]  $\leftarrow$  obj_values;
12    end
13  end
14  v = mean(cost_per_rollout, axis=1);
15 end
16 m_list, obj_values  $\leftarrow \text{SICIF}(m, n + 1, [(g_{0,i})_{i \in [n]} \text{ and } v])$ ;
17 return m_list, obj_values, v;
```

We should take some time to understand why this algorithm works. The v variable defined on line 3 represents the estimate of the value function at the next time step, discounted by the possibility that the current time step is the final one. So at iteration t of the for-loop, v is the estimate of $(1 - \rho_t)V_{t+1}(\cdot)$, where $v[z] \approx (1 - \rho_t)V_{t+1}(z)$ for $z = 0, 1, \dots, m$. If we treat v as a function, we have that it is monotonic decreasing and satisfies $v[1] - v[0] < v[2] - v[1] < \dots < v[m] - v[m-1]$. Because of this property, we can treat v as one of the convex functions f_i found in the **SICIF** problem. In lines 10 and 16, we use Algorithm 1 to solve the min problems seen in 19 and **MP** respectively. Because the `obj_values` array returned by Algorithm 1 is also monotonic decreasing and “convex”, we have that v always maintains its convexity. Therefore, calls to Algorithm 1 in lines 10 and 16 always find the global optimum.

If we were to assume that T is actually fixed, and that $n_t = n$ is also fixed at each time step, we would have that Algorithm 2 terminates in $O(RT(m+n)\log n)$ time.

2.2 The $T \sim \text{Geometric}(\rho)$ Case

I’ll end this report by discussing the interesting special case where $T \sim \text{Geometric}_0(\rho)$, where Geometric_0 is the version of the geometric distribution that puts positive mass on $T = 0$. The interpretation behind this assumption is that at the beginning of each time step $t = 0, 1, \dots$, we sample $D_t \sim \text{Bernoulli}(\rho)$. If $D_t = 1$, then the current time step t is the final one. This assumption therefore implies that $\rho_t = \rho$ for all t , and that the current time step t is not predictive of how many projectile barrages are yet to come.

The multi-period problem then becomes

$$\begin{aligned} \min_{m_1, \dots, m_n} & \left\{ \sum_{i=1}^n (\bar{c}_i (1-p)^{m_i} + c_{DPI} \cdot m_i) + (1-\rho)V(m - \sum_{i=1}^n m_i) \right\} \\ \text{s.t.} & \sum_{i=1}^n m_i \leq m, \quad m_i \in \mathbb{N} \cup \{0\}, \end{aligned} \quad (\text{GMP})$$

where

$$V(\mu) = \mathbb{E}_{n_t} \mathbb{E}_{(\Delta_{t,i})_{i \in [n_t]}} \left[\min_{(m_{t,i})_{i \in [n_t]}} \left\{ \sum_{i=1}^{n_t} (\bar{c}_{t,i} (1-p)^{m_{t,i}} + c_{DPI} \cdot m_{t,i}) + (1-\rho)V(\mu - \sum_{i=1}^{n_t} m_{t,i}) \right\} \right]. \quad (20)$$

Assuming that T follows a geometric distribution simplifies the multi-period problem, because we can simply do value iteration on equation 20 to get an estimate of $V(\cdot)$. Once that is done, we can find an approximate solution to **GMP**.

Algorithm 3: Value Iteration with Monte Carlo

Data: $m \geq 0$, $\text{rho} \in (0, 1)$, $R > 0$, $\text{max_iter} \in \mathbb{N}$

```

1 v  $\leftarrow$  zeros( $m+1$ );
2 gamma  $\leftarrow$   $1 - \text{rho}$ ;
3 for iter = 1 to max_iter do
4   cost_per_rollout  $\leftarrow$  zeros( $(m+1, R)$ );
5   for  $r = 1$  to  $R$  do
6     Sample  $n_t \sim P(n_t)$ ;
7     Generate  $g_{t,i}(\cdot)$  for  $i \in [n_t]$ ;
8      $\_, \text{obj\_values} \leftarrow \text{SICIF}(m, n_t + 1, [(g_{t,i})_{i \in [n_t]} \text{ and } \text{gamma} * \mathbf{v}])$ ;
9     cost_per_rollout $[:, r] \leftarrow \text{obj\_values}$ ;
10  end
11   $\mathbf{v} = \text{mean}(\text{cost\_per\_rollout}, \text{axis}=1)$ ;
12 end
13 return v;

```

We have that Algorithm 3 returns an estimate of the value function $\hat{V}(z)$ for $z = 0, 1, \dots, m$. We can then approximately solve **GMP** using Algorithm 1 where $\hat{V}(\cdot)$ is used instead of $V(\cdot)$. Note that $\hat{V}(\cdot)$ is monotonic decreasing and convex. It's also important to note that in Algorithm 3, the array $\mathbf{v}$ represents an estimate of $V(\cdot)$, whereas in Algorithm 2 it represents $(1 - \rho_t)V_t(\cdot)$. Unlike in Algorithm 2, Algorithm 3 does the discounting explicitly by multiplying the array $\mathbf{v}$ by the scalar constant **gamma** (which is equal to $1 - \text{rho}$) on line 8.

3 Experiments

3.1 Dynamic Programming with Monte Carlo Experiment

In this example we use Algorithm 2 in order to solve a multi-period problem of the form **MP**. Let $T \sim \text{Unif}[6, 12]$ (i.e., T is an integer uniformly distributed over $6, 7, \dots, 12$), and let $n_t \sim \text{Poisson}(\lambda)$ where $\lambda = 5$. In this example we assume there are $l = 5$ zones with costs $\{40, 60, 100, 300, 500\}$. The probabilities associated with projectile i at time t hitting each zone is given by $\Delta_{t,i} \sim \text{Dirichlet}(\alpha)$ where $\alpha = [\frac{1}{5} \ \frac{1}{5} \ \frac{1}{5} \ \frac{1}{5} \ \frac{1}{5}]^T$. Lastly, we set the number of rollouts to be done in Algorithm 2 to $R = 500$.

At time step $t = 0$ (i.e., the current time step) there are 9 projectiles, where the expected cost imposed by the i^{th} one is given by Table 1. For example, $\bar{c}_i = 200$ for $i = 3$. We assume that the cost of launching an interceptor is $c_{DPI} = 50$. Table 2 gives the optimal number of interceptors $m_1, \dots, m_9$ allocated to each projectile as given by Algorithm 2. This is done for varying levels of m , the total number of interceptors we have at our disposal. We see that as m decreases, Algorithm 2 uses less interceptors in the current time step in order to ration for the future.

Projectile	1	2	3	4	5	6	7	8	9
$\bar{c}_i$	421	80	200	50	450	175	60	302	427

Table 1: The expected costs $\bar{c}_i$ associated with each projectile.

m	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	Remaining
100	3	0	1	0	3	1	0	2	3	87
50	2	0	1	0	2	1	0	2	2	40
25	2	0	0	0	2	0	0	1	2	18
10	1	0	0	0	1	0	0	0	1	7

Table 2: The number of interceptors allocated to each projectile for varying levels of m .

Figure 1 below visualizes $\hat{V}_1(z)$, the estimate of $V_1(z)$, for $z = 0, \dots, m$. For the graph we have m set to 25. Technically it actually visualizes the estimate of $(1 - \rho_1)V_1(z)$, but in this case $\rho_1 = 0$.

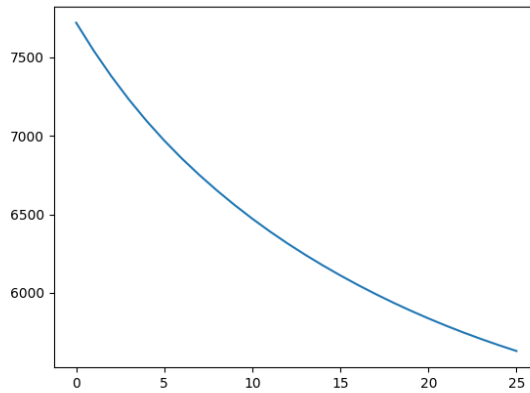


Figure 1: The estimate of $V_1(z)$ for when $m = 25$.

3.2 Value Iteration with Monte Carlo Experiment

In this example we use Algorithm 3 in order to estimate the value function $V(\cdot)$ found in GMP. We assume $T \sim \text{Geometric}_0(\rho)$ where $\rho = 0.1$. Just as in Experiment 3.1, we assume that $n_t \sim \text{Poisson}(\lambda)$ where $\lambda = 5$, $c_{DPI} = 50$, and that there are $l = 5$ zones with costs $\{40, 60, 100, 300, 500\}$. Once again the probabilities associated with projectile i at time t hitting each zone is given by $\Delta_{t,i} \sim \text{Dirichlet}(\alpha)$ where $\alpha = [\frac{1}{5} \ \frac{1}{5} \ \frac{1}{5} \ \frac{1}{5} \ \frac{1}{5}]^T$. In this experiment we set the number of rollouts to be done in Algorithm 3 to $R = 50$, and we set $m = 100$. Figure 2 below shows the estimate of the value function $V(z)$ at $z = 0, \dots, m$.

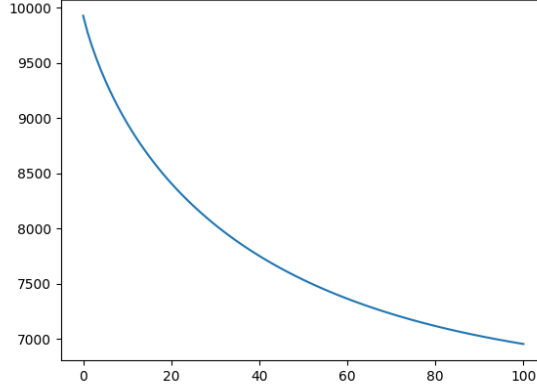


Figure 2: The estimate of $V(z)$ for $z = 0, \dots, 100$.

The value function estimate $\hat{V}(\cdot)$ can then be used to solve GMP, where $\hat{V}(\cdot)$ is used in place of $V(\cdot)$.